# 數位電路實驗結報二

# – 數位顯示器 –

**Group 2: B11303052 蘇盈翰 (Right), B11702091 張容爾 (Middle), B10901144 王俊皓 (Left)**

April 29, 2026

# 1 實驗原理

## 1.1 像素獨立控制的原理

若要將 LED 顯示器中任一像素點亮，最直觀的作法是讓每一顆 LED 都擁有獨立的控制電位。對於一個 $m \times n$ 的 LED 矩陣而言，獨立控制將需要 $m \cdot n$ 條控制線。以本次實驗的 $8 \times 8$ LED 矩陣為例，共需 64 條控制線，若擴展至 $2868 \times 1320$ 像素的手機螢幕，更需高達 3,785,760 條控制線，實為沒有效率，且 ESP32 受限於腳位數量與用途，實際可自由運用的腳位約僅 20 餘個，不足以獨立控制 LED 的每個像素。

為了降低控制線數量，最常見的作法是採用**共陰共陽** (common cathode / common anode) 矩陣。在此架構下，同一列的所有 LED 共用同一條陰極（或陽極），同一行則共用陽極（或陰極），使得控制線數從 $m \cdot n$ 降為 $m + n$。對 $8 \times 8$ 矩陣而言，控制線數從 64 條減少為 16 條。

但共陰共陽架構帶來一項重要限制：**無法在單一瞬間任意點亮非全矩形的 LED 組合**。舉例說明，若希望同時點亮位於 (列 1, 行 1) 與 (列 2, 行 2) 的兩顆 LED，則必須將列 1、列 2 同時拉低（陰極），並將行 1、行 2 同時拉高（陽極）。然而此操作會使 (列 1, 行 2) 與 (列 2, 行 1) 處的 LED 也同時被點亮，產生四顆 LED 全亮，無法達到預期效果。

## 1.2 時域多工

為解決上述限制，本實驗採用**時域多工** (Time-Division Multiplexing, TDM) 搭配人眼視覺暫留 (Persistence of Vision) 特性，將控制線最少化並達成 LED 陣列的像素顯示效果。

時域多工是一種將單一傳輸通道於時間軸上切分給多個訊號輪流使用的技術，每個訊號只在自己被分配到的時間片段內傳送或被接收，但因切換速度極快，從整體上看就像所有訊號同時存在，在本實驗中，我們將其應用於 LED 矩陣的顯示控制，每一個瞬間僅點亮單一列的 LED，並以可變電阻調整閃爍頻率。當切換頻率高於人眼可分辨的閾值，人眼便會因視覺暫留而將時間上分離的瞬時畫面整合為一幅連續且靜止的影像。

## 1.3 移位暫存器 IC 74595

為進一步壓縮 ESP32 所需腳位，本實驗使用**移位暫存器** (Shift Register) IC 74595，以達成時域多工的效果。74595 為一 8 位元串入並出 (Serial-In, Parallel-Out, SIPO) 暫存器，其核心腳位包含：

- **SER (Data)**：串列資料輸入端。
- **SRCLK (Clock_transfer)**：每次上升緣會將 SER 上的訊號讀入暫存器，並將既有資料依序往後推一格。
- **RCLK (Clock_show)**：每次上升緣將內部暫存器內容鎖存至 8 個並列輸出 ($Q_A - Q_H$)。
- $\boldsymbol{Q_{H'}}$：暫存器最末位之串列輸出，可接至下一顆 74595 的 SER 以實現級聯。

本次實驗使用兩顆 74595 ：一顆控制 LED 矩陣的 8 條陽極，另一顆控制 8 條陰極，僅需單一 SER 即可連續送入 16 位元資料。如此一來，原本 16 條控制線被壓縮為 3 條：

$$8 \times 8 = 64 \xrightarrow{\text{共陰共陽}} 8 + 8 = 16 \xrightarrow{\text{74595 級聯}} 3 \tag{1}$$

# 2 實驗架設與步驟

## 2.1 硬體電路架設

1. **元件**：$8 \times 8$ 共陽 LED 矩陣 $\times$ 1、74HC595 移位暫存器 $\times$ 2、ESP32 開發板 $\times$ 1、330 Ω 限流電阻 $\times$ 8、10 kΩ 可變電阻 $\times$ 1、麵包板與杜邦線若干、USB 供電線。

2. **電源與接地**：將 ESP32 之電源與 GND 接至麵包板兩側電源軌，並完成兩顆 74595 之電源、接地連接。

3. **74595 級聯**：將 74595-1（控制陽極）之 $Q_{H'}$ 接至 74595-2（控制陰極）之 SER，並使兩顆 IC 之 SRCLK、RCLK 共接，與 ESP32 共用相同時脈。

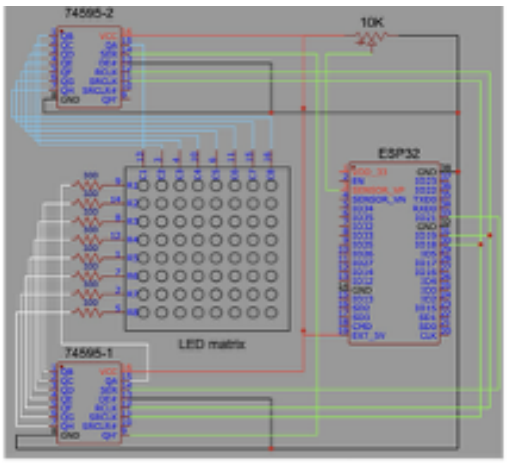
4. **ESP32 控制訊號接線**：
   - IO21 → SER（Data）
   - IO18 → SRCLK（Clock_transfer）
   - IO19 → RCLK（Clock_show）
   - SENSOR_VP → 10 kΩ 可變電阻中央接點，用以讀取分壓並調整刷新頻率

5. **LED 矩陣連接**：陽極側 8 支腳位各串接一顆 330 Ω 限流電阻後接至 74595-1 之 $Q_A - Q_H$；陰極側 8 支腳位直接接至 74595-2 之 $Q_A - Q_H$，對應關係如表 1。

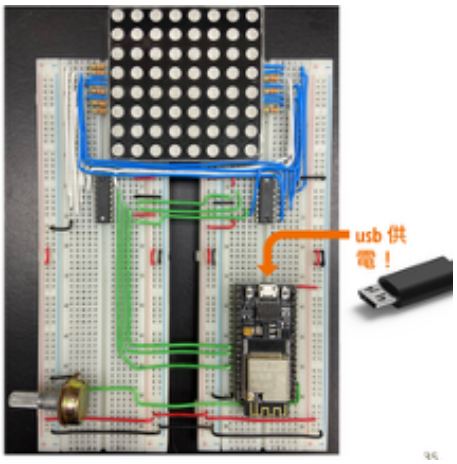
| **74595-1（陽極）** | **LED matrix pin** | **74595-2（陰極）** | **LED matrix pin** |
|---|---|---|---|
| $Q_A$ | 9 | $Q_A$ | 13 |
| $Q_B$ | 14 | $Q_B$ | 3 |
| $Q_C$ | 8 | $Q_C$ | 4 |
| $Q_D$ | 12 | $Q_D$ | 10 |
| $Q_E$ | 1 | $Q_E$ | 6 |
| $Q_F$ | 7 | $Q_F$ | 11 |
| $Q_G$ | 2 | $Q_G$ | 15 |
| $Q_H$ | 5 | $Q_H$ | 16 |

Table 1: 74595 輸出腳位與 LED 矩陣對應關係

完整之電路邏輯圖與實作照片如圖 1a 與圖 1b 所示。




(a) LED 陣列實驗之電路邏輯




(b) LED 陣列實驗之電路實作

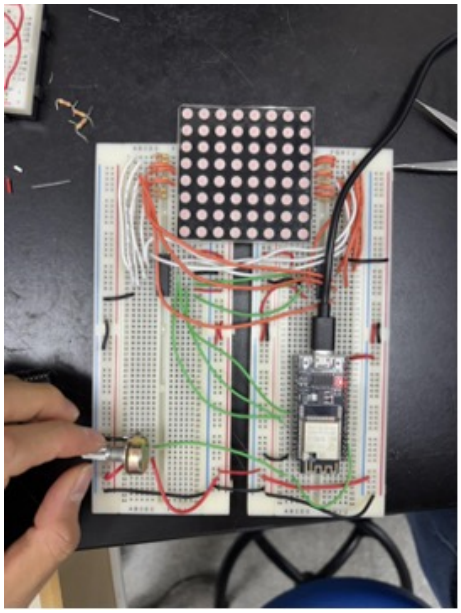
Figure 1: LED 陣列實作圖

## 2.2 實驗一：LED 陣列

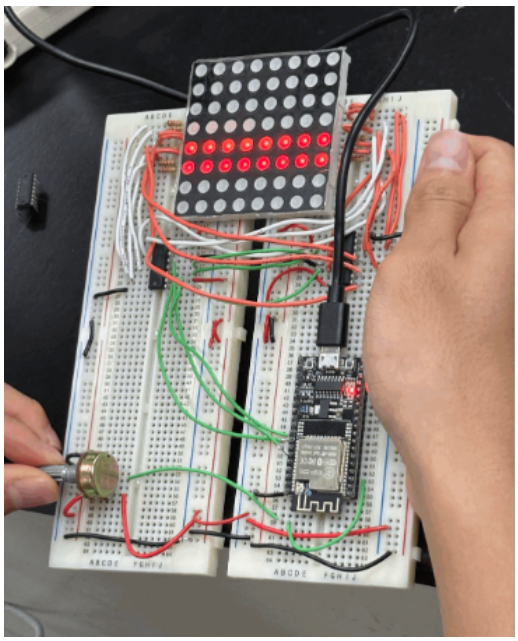
1. **韌體燒錄**：透過 USB 將 ESP32 連接至電腦，於 Arduino IDE 中選取 NodeMCU-32S 板型，將 `Lab06_Experiment_1_led-scanning.ino` 燒錄至 ESP32。該程式以時域多工方式，逐列輸出全亮資料至 LED 矩陣。

2. **動態頻率觀察**：通電後旋轉可變電阻，觀察 LED 矩陣之顯示變化。當刷新頻率低時，可清楚看見單一列依序亮起的列掃描現象；當刷新頻率高時，由於人眼視覺暫留，整個矩陣看似全亮。

3. **第一週除錯紀錄**：我們於第一週實驗中發現 LED 矩陣呈現恆亮狀態（如圖 2a），僅可調整整體亮度而無法觀察到列掃描閃爍。經助教協助除錯後，確認問題出在其中一條接地線因接觸不良而需透過手部接觸方能導通；重新固定接地線後，於第二週實驗順利重現預期之掃描動畫，如圖 2b 所示。此外，當可變電阻轉至最低阻值時，刷新頻率反而出現異常變慢的現象，推測與該可變電阻實際阻值未達標稱 10 kΩ 有關，極端值處之分壓行為偏離預期。



(a) 第一週實驗：LED 矩陣呈現恆亮，無法觀察列掃描現象



(b) 除錯後成功觀察到不同刷新頻率下的列掃描閃爍

Figure 2: 實驗一

## 2.3 實驗二：LED 陣列動畫顯示

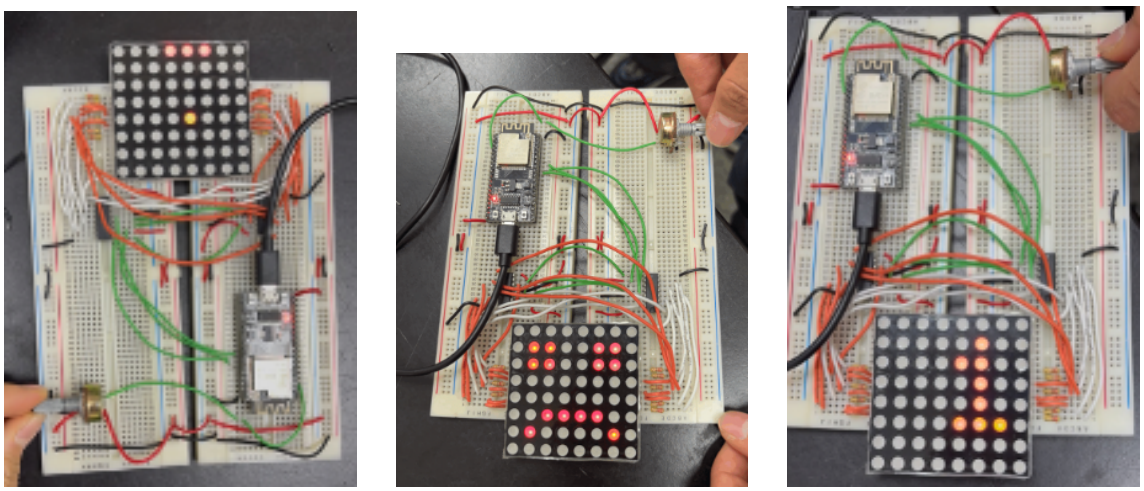
1. **圖案編輯**：使用線上工具 LED Matrix Editor[1] 繪製欲顯示之圖案，工具會輸出對應之 64-bit 十六進位陣列。

2. **韌體修改**：將 `Lab06_Experiment_2_running-8x8.ino` 燒錄至 ESP32，並以上述工具產生之陣列替換程式中之 `image[]`。程式內部仍以時域多工方式逐列掃描，但每列之資料改由陣列指定，從而可顯示任意靜態圖案；若於 `loop()` 中依序切換多個陣列，則可形成動畫效果。

3. **互動式 Emoji Pong 遊戲實作**：在動畫顯示之基礎上，我們跟 AI 協作，進一步擴充程式，將可變電阻之分壓讀值映射為畫面下緣反彈板之水平位置，並加入小球之物理運動邏輯（如速度、邊界反射等）。每次反彈板未接到小球時，遊戲重置，一共共經三階段擴充：

   - **初階版**（圖 3a）：基本的 pong 玩法，反彈板未接到即立即重置。
   - **進階版**（圖 3b）：失敗時於矩陣上顯示哭臉圖案 3 秒後再重置。
   - **最終版**（圖 3c）：失敗後以跑馬燈顯示本局得分，再開始新局；同時每隔 30 秒線性提升小球速度，增加遊戲難度。

[1] https://xantorohara.github.io/led-matrix-editor/



(a) 初階版 (b) 進階版 (c) 最終版

Figure 3: Emoji Pong 三階段版本對照

# 3 實驗結果與分析

## 3.1 時域多工與視覺暫留驗證

在本實驗中，我們成功利用兩顆 74HC595 移位暫存器級聯，將 ESP32 的 3 個 GPIO 腳位擴展至可控制 $8 \times 8$ LED 矩陣。實驗結果顯示：

- **低頻掃描階段**：當可變電阻調整至高阻值時，輸入 ESP32 的電壓經 ADC 轉換後設定較長的延時，此時可清晰觀察到 LED 矩陣以「列」為單位，由上而下逐列亮起。這證實了時域多工 (TDM) 的運作機制，即任何瞬間僅有一列陰極被拉低 (Low Level)。
- **高頻顯示階段**：隨著可變電阻旋轉降低阻值，掃描頻率提升。當刷新頻率超過約 60 Hz 時，由於人眼視覺暫留效應，逐列掃描的閃爍感消失，轉化為穩定的全亮畫面或靜態圖案。

## 3.2 Emoji Pong 遊戲邏輯與動態顯示

在實驗二的動畫顯示部分，我們成功實作了互動式遊戲。分析如下：

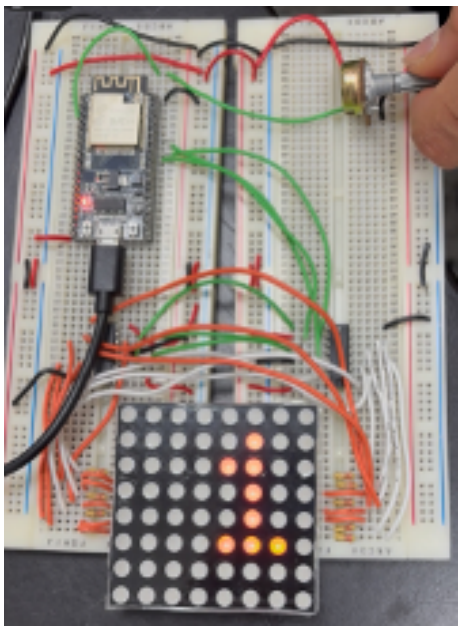


Figure 4: 最終版 Emoji Pong 於 $8 \times 8$ 矩陣之顯示效果

1. **映射精準度**：透過將可變電阻的類比訊號映射 (Mapping) 至 0–5 的區間（考慮到反彈板長度為 3 像素），玩家能平滑地控制畫面底部的擋板，反應延遲極低，顯示移位暫存器在高頻運作下的資料傳輸效率足以支撐即時互動。

2. **難度曲線分析**：最終版程式加入每 30 秒線性提升小球移動速度的邏輯。實驗發現，當小球速度過快時，受限於 $8 \times 8$ 的解析度，小球的軌跡會出現明顯的跳躍感（非連續移動），這說明了低解析度顯示器在呈現高速動態物體時的物理限制。

3. **得分跑馬燈**：透過切換不同的 `image[]` 陣列並結合位移邏輯，我們成功在有限的空間內顯示兩位數以上的得分，達成資訊傳達的功能。

### 3.3 誤差與除錯分析

針對實驗過程中出現的異常現象進行探討：

- **接地接觸不良**：第一週出現的恆亮現象，經分析是因為接地線未完全導通，導致電流尋找其他路徑或使電位浮動，造成掃描邏輯失效。

- **可變電阻極值異常**：觀察到在電阻最小端刷新率反而變慢，推測是因為 ADC 讀取值在極值附近因雜訊波動，觸發了程式碼中預設的保護機制或溢位（Overflow），導致延時函數 (`delay`) 的參數異常。

## 4 問題討論

### 4.1 為什麼 LED 矩陣的腳位會這麼亂，有什麼好處嗎？

站在設計者角度的話，會根據配合驅動 IC 的腳位，進而去設定 8*8 的 LED 陣列腳位，就好像 dynamic duo 一樣，產品一套賣出，對買者有良好的使用一致性體驗，賣家和設計者也能藉此商機多賺一筆。而 LED 陣列有一個最主要的配合驅動 IC，就是 MAX7219。要問 LED 陣列為什麼這麼亂，就得追本溯源，問 MAX7219 的腳位為何？查詢後，果然 MAX7219 腳位看起來亂，是因為它不是照

「數字順序」排，而是照「電流、晶片內部 layout、封裝腳位、七段顯示器/LED matrix 走線」排。而你若細看 MAX7219，他有 DIG 和 SEG 兩種 port，DIG 選擇 8 個 column 中哪些要亮，SEG 選擇每個 colunm 中，哪幾顆 LED 要亮。了解此，進而去細究 DIG 的腳位配置，觀察 sequence04623751 的規律，這是個 Gray code like sequence，若要用 Gray code 去理解 IC 腳位設計的話，Gray code cube 使得腳位間的電訊號更好地利用 SIPO 這特點。這使得。而 Gray code 有一個最主要的 encode 特徵，相鄰兩點只差一個 bit variance。舉例來說，在 Gray code Cube 上，000 的鄰居有，001,010,100，也就是說腳位 1 的鄰居是腳位 1,2,4。這種設計，優化了 decoder/driver layout 設計。接下來要了解的 SEG 腳位設計，這種設計是為了迎合七段顯示器的的掃描順序，AFBGCED，這樣的腳位安排能將此 IC 更好地融合到七段顯示器上。

## 4.2 電視或電腦等更多像素的螢幕是怎麼控制的，與今天電路的相似與相異之處？

電視或電腦螢幕雖然像素數量比今天的 LED 矩陣多非常多，但基本概念其實相似：它們都是把畫面切成一個二維矩陣，用「列」和「行」的方式來控制每個像素的位置。
今天的 LED 矩陣是利用 row 和 column 交會的位置來點亮 LED，例如先選擇某一列，再決定這一列中的哪些欄要亮；電視和電腦螢幕也是依照類似的掃描概念，一列一列地更新畫面。

以電腦螢幕為例，GPU 會先在記憶體中產生一張完整的畫面，這張畫面叫做 framebuffer，裡面記錄了每個像素的顏色資訊（如紅、綠、藍數值）。接著，GPU 會透過 HDMI、DisplayPort 或筆電內部的 eDP，把這些影像資料高速傳送到螢幕。

螢幕內部的 timing controller (TCON) 會接收這些資料，並控制面板內部的 row driver 和 column driver，把資料送到正確的位置。

如果是 LCD 螢幕，每個像素本身不會發光，而是透過液晶控制背光能否通過。LCD 面板中每個像素旁邊都有一個很小的 TFT 電晶體，可以把它想成一個微小開關。

螢幕會先選擇某一列，打開那一列所有像素的 TFT，然後 column driver 把每個像素需要的電壓送進去。這個電壓會暫存在像素旁邊的小電容中，讓液晶在下一次更新前維持同樣的透光程度。也就是說，LCD 的掃描不是「掃到才亮」，而是「掃到時寫入資料，之後像素自己保持狀態」。

如果是 OLED 或 AMOLED 螢幕，每個像素本身會發光，所以它更像是超大型、超精密的 LED 矩陣。

不過它也不是用簡單的被動掃描方式，因為像素太多，如果每一列只在掃描到時才亮，亮度會太低、瞬間電流會太大。因此 OLED 螢幕通常也使用 active matrix 結構，每個像素附近都有 TFT 和儲存電容，用來記住亮度資料，並在一整個 frame 期間持續控制 OLED 發光。

這和今天的 LED 矩陣電路有幾個相似處：
1. ** 矩陣結構 **：都可以用 row 和 column 來定位像素。
2. ** 掃描概念 **：並非每個像素獨立連線，而是透過分時掃描的方式更新。

3. ** 驅動 IC**：都需要專用晶片處理大量腳位，如 MAX7219 之於 8x8 矩陣，TCON 之於大螢幕。

但兩者也有很大的不同：

今天的 LED 矩陣比較接近「被動矩陣控制」，LED 通常只有亮或不亮；而現代螢幕像素達數百萬個，需要複雜的灰階、電壓與電流控制。
最關鍵的區別在於：LED 矩陣通常是掃到才亮，而現代螢幕多半是「主動矩陣」(Active Matrix)，掃描時只是把資料寫入電容，之後像素會靠 TFT 維持狀態直到下一幀。

因此，今天的 LED 矩陣可以看作是現代螢幕控制原理的「簡化模型」。兩者核心都是矩陣與掃描；不同的是，現代螢幕為了達成彩色與高解析度，加入了 GPU、framebuffer、TFT 與類比驅動電路，讓整個畫面能更穩定且精細地顯示。

## 4.3 最後感謝助教，晏德心

感謝助教幫我們 debug LED 陣列 ground 沒接好的問題，讓我知道麵包板哪些 tricky part 要注意，但最重要的是，讓我學到 debug 的方法，就是根據每個點位的電位特性把玩，認真下手研究的精神。